PREDICTIVE MAINTENANCE MODEL FOR MANUFACTURING MACHINES

A PROJECT REPORT

Submitted by

NAVEEN D

61782323401033

in partial fulfillment for the award of the degree

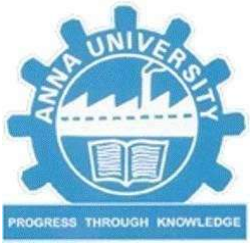
of

MASTER OF COMPUTER APPLICATIONS

IN

SONA COLLEGE OF TECHNOLOGY, SALEM

(Autonomous)



ANNA UNIVERSITY : CHENNAI 600 025

APRIL 2025

SONA COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that the project entitled

PREDICTIVE MAINTENANCE MODEL FOR MANUFACTURING MACHINES

is the approved record of project work done by

NAVEEN D

61782323401033

in the partial fulfillment of the requirements for the award of the degree of Master of Computer Applications from Sona College of Technology, Salem during the period Jan 2025 - April 2025.

……...……	…….....
Project Guide	**Head of the Department**
[Dr. J. Suji Priya MCA, Ph.D.,]	**[Dr. T. Padma MCA, Ph.D.,]**

Submitted for the Project Viva-Voce examination held on …….

……	…….
Internal Examiner	External Examiner

BONAFIDE CERTIFICATE

Certified that this project report titled **"Predictive Maintenance Model for Manufacturing Machines"** is the bonafide work of **NAVEEN D , Register Number: 61782323401033** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report. On the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Project Guide

[Dr. J. Suji Priya MCA, Ph.D.,]

Head of the Department

[Dr. T. Padma MCA, Ph.D.,]

ABSTRACT

Predictive Maintenance has emerged as a crucial area of research in smart industry and the Industrial Internet of Things (IIoT), aiming to anticipate machine failures and minimize unplanned downtime. This project focuses on developing an intelligent system capable of predicting the Remaining Useful Life (RUL) of machinery, classifying maintenance status, and detecting anomalies based on real-time sensor data. The system processes features such as temperature, vibration, and pressure, which are preprocessed using data cleaning and normalization techniques to ensure accuracy. Machine learning models including Random Forest Regressor, Random Forest Classifier, and K-Means Clustering are employed to perform predictive analytics and anomaly detection. These models are trained and evaluated using historical machine datasets, enabling the system to learn failure patterns and generate reliable predictions. The complete system is deployed via a user-friendly Streamlit web interface that supports data input, result visualization, and real-time monitoring. By providing early warnings and actionable insights, the system significantly reduces maintenance costs and enhances overall equipment efficiency. The solution is modular, scalable, and adaptable to various industrial setups. It also contributes to sustainability by extending machine lifespan and minimizing energy waste. This project demonstrates the practical impact of AI in predictive maintenance and its potential in transforming industrial operations.

HACKVEDA LIMITED

011-27297608
hr@hackveda.in

C-25, First Floor,
Sector-8, Noida, UP,
201301

H-3/60, III Floor,
Sector-18, Rohini, Delhi -
110089

615-50, Dunsheath Way,
Markham, ON L6B 1N3

Date of Issue: 20 February 2025

Internship Completion Letter - Data Analyst Internship

Candidate Name: NAVEEN D
Candidate Email: naveen.23cap@sonatech.ac.in
Candidate Mobile: 8903477545

Dear Concerned,

I hereby certify the candidate was employed as **Intern** with our company Hackveda Limited during the period.

Internship Start Date: 16 November 2024

Internship Completion Date: 20 February 2025

He/She started working with Hackveda Limited as an Intern and with sincere efforts & excellent performance was moved to the position of professional responsibilities.

For the duration of working for us, he/she demonstrated as a diligent and truthful person. His/Her interpersonal skills are outstanding and he/she has been very helpful and has been highly appraised by his/her Managers.

We wish him/her success in his/her future efforts.

Yours sincerely,

Devanshu Shukla

Director, Hackveda Delhi

Hackveda Limited, Canada

011-27297608, +91-9654825370

ACKNOWLEDGEMENT

First and foremost I concede the surviving presence and the flourishing refinement of The **Almighty** for his concealed hand yet substantial supervision all through the project.

I am extremely grateful to **Sri. C. Valliappa,** Chairman, **Sri. Chocko Valliappa** and **Sri. Thyagu Valliappa,** Vice-Chairmen of Sona College of Technology, Salem, for providing me the opportunity and enough facilities to complete my project work.

I record my deep sense of gratitude and will remain obliged to **Dr. S. R. R. Senthil Kumar**, Principal of Sona College of Technology, Salem, for his valuable help and suggestions to complete my project.

I also thank **Dr. T. Padma, MCA., Ph.D**., Head of the Department, Master of Computer Applications for her encouragement, valuable suggestions and support in completion of this project.

I express my sincere thanks to my Project Coordinator and Guide **Dr. J. Suji Priya MCA, Ph.D., Assistant Professor** of our department, for her timely suggestions, valuable guidance to carry out this project as a grand success.

I would like to thank all **technical and non-technical staff members** of the Department of Master of Computer Applications for the kind cooperation and support rendered in making my project success.

Lastly,I am deeply thankful to my **parents** for their constant support, encouragement, and motivation, which played a vital role in the successful completion of this project.

LIST OF FIGURES

S.NO	NAME OF THE FIGURE	PAGE NO.
5.1	System Architecture	13
5.2	Class Diagram	14
5.3	Use Case Diagram	15
6.1	Sample Dataset	18
6.2	Input for Prediction	22
6.3	Prediction	22
6.4	Chart for Sensor Readings	23
6.5	Chart for Sensor Vs Operational Hours	24
6.6	RUL Visualisation	24
A.2.1	Home Page	38
A.2.2	Sample Dataset	38
A.2.3	Random Input Generation	39
A.2.4	User Inputs	39
A.2.5	Prediction with Anomaly	40
A.2.6	Prediction without Anomaly	40
A.2.7	Histogram chart for Sensor readings	41
A.2.8	Scatter Plot for Sensor Readings vs Opr. Hours	41
A.2.9	Line chart for RUL vs Operational Hours	41

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	Abstract	iv
	Acknowledgement	vi
	List of Figures	vii
1	**Introduction**	
	1.1 Overview of the project	1
	1.2 Objective of the project	2
	1.3 Literature Review	3
2	**System Analysis**	
	2.1 Problem Definition	4
	2.2 Existing System	5
	2.3 Proposed System	6
3	**System Study**	
	3.1 Feasibility Study	8
	3.1.1 Economic Feasibility	8
	3.1.2 Technical Feasibility	8
	3.1.3 Performance Feasibility	9
4	**System Requirements**	
	4.1 Hardware Requirements	10
	4.2 Software Requirements	10
	4.3 Software Description	11
5	**System Design**	
	5.1 System Architecture	13
	5.2 Class Diagram	14

	5.3 Use Case Diagram	15
6	**Proposed Algorithm Implementation**	
	6.1 Project Description	16
	6.2 Modules	17
	6.3 Module Description	17
	6.3.1 Dataset	17
	6.3.2 Data Preprocessing	19
	6.3.3 Training and Evaluation	20
	6.3.4 Prediction	21
	6.3.5 Visualisation	23
7	**Results and Discussion**	25
8	**System Testing**	
	8.1 Testing	26
	8.2 Unit Testing	26
	8.3 Integration Testing	26
	8.4 Validation Testing	27
	8.5 Testing Report	27
9	**Conclusion and Future Enhancements**	
	9.1 Conclusion	28
	9.2 Future Enhancements	28
	Appendix	
	A.1 Source Code	30
	A.2 Screenshots	38
	References	42

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Predictive Maintenance for Industrial Equipment is a crucial advancement in industrial operations, aimed at minimizing unexpected machinery failures, optimizing maintenance schedules, and improving overall efficiency. Equipment breakdowns can lead to significant operational disruptions, financial losses, and safety hazards. Traditional maintenance methods, such as reactive or time-based approaches, often fail to detect early signs of failure, making predictive maintenance a more effective solution. Predictive maintenance relies on machine learning models and real-time sensor data to monitor the health of industrial machinery. It enables industries to predict potential failures before they occur, reducing downtime and preventing costly breakdowns. The World Economic Forum (WEF) and other industrial research organizations highlight that predictive maintenance significantly enhances equipment reliability and extends asset lifespan.

In this project, sensor readings and operational data are analysed to determine the Remaining Useful Life (RUL) of machines, using a Random Forest Regressor for precise failure prediction. A Random Forest Classifier identifies whether maintenance is required, while K-Means Clustering is used for anomaly detection, spotting unusual sensor behaviour that may indicate early-stage malfunctions. With unplanned downtime costing industries billions annually, predictive maintenance plays a crucial role in ensuring uninterrupted operations. This project includes a Streamlit-based interactive dashboard, which allows users to visualize real-time equipment health, detect anomalies, and receive predictive maintenance alerts. Data preprocessing techniques, such as cleaning, normalization, and feature selection, ensure model accuracy and reliability. Predictive maintenance is transforming industrial sectors by reducing costs, improving safety, and enhancing operational efficiency. By adopting AI-driven predictive maintenance, industries can transition from a reactive approach to a proactive strategy, ensuring long-term sustainability and seamless industrial operations.

1.2 OBJECTIVES OF THE PROJECT

- To develop an AI-driven predictive maintenance system that forecasts potential machinery failures before they occur, reducing unplanned downtime and operational disruptions.
- To implement machine learning models such as Random Forest Regressor, Random Forest Classifier, and K-Means Clustering for failure prediction, anomaly detection, and maintenance classification.
- To analyze historical sensor data and operational metrics to predict the Remaining Useful Life (RUL) of industrial equipment, ensuring timely maintenance actions.
- To integrate a real-time, interactive dashboard using Streamlit, allowing users to visualize sensor data, equipment health status, and predictive insights for proactive decision-making.
- To enhance industrial efficiency and safety by transitioning from traditional reactive maintenance to data-driven predictive maintenance, reducing maintenance costs and improving machine reliability.
- To apply advanced data preprocessing techniques, such as data cleaning, normalization, and feature engineering, to improve the accuracy and efficiency of machine learning models.
- To enable early detection of equipment anomalies using unsupervised learning techniques like K-Means Clustering, preventing unexpected failures and improving system resilience.
- To optimize resource utilization and extend machine lifespan by ensuring maintenance actions are performed only when necessary, minimizing operational costs and maximizing asset performance.

1.3 LITERATURE REVIEW

Predictive maintenance (PdM) has emerged as a key technology in Industry 4.0, leveraging machine learning, deep learning, and IoT-driven analytics to minimize unplanned downtimes, optimize asset performance, and enhance industrial efficiency. Several research studies have explored predictive maintenance models, focusing on their implementation, benefits, and challenges.

Paolanti et al. (2018) proposed a machine learning-based predictive maintenance system for industrial automation, integrating sensor data and predictive algorithms to improve fault detection accuracy. Their research demonstrated how predictive analytics could shift traditional maintenance strategies from reactive to proactive, reducing operational costs and equipment failures[1]. Abbasi (2021) conducted a study on the application of machine learning techniques for industrial machinery maintenance. His research utilized classification and regression models, including Random Forest and Support Vector Machines (SVMs), to predict failure probabilities. The study emphasized the role of real-time data streaming and cloud-based computing in enabling scalable predictive maintenance systems[2].

Khan and Yairi (2018) reviewed deep learning applications in system health management, highlighting the advantages of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) in analyzing large volumes of sensor data. Their work showcased how deep learning surpasses traditional predictive maintenance approaches by providing more accurate anomaly detection and failure prediction in industrial systems[3]. Zonta et al. (2020) presented a systematic literature review on predictive maintenance in Industry 4.0, categorizing existing methodologies such as condition-based maintenance (CBM), data-driven techniques, and hybrid models. Their study found that the integration of IoT, AI, and cloud computing significantly enhances predictive maintenance capabilities, allowing for real-time monitoring and automated failure prediction[4].

Sang et al. (2021) developed a predictive maintenance model for flexible manufacturing systems, focusing on data-driven decision-making and intelligent maintenance planning. Their research demonstrated how machine learning algorithms could predict failure patterns, optimize maintenance schedules, and minimize production disruptions. The study also discussed the importance of manufacturing execution systems (MES) integration in achieving automated maintenance workflows[5].

CHAPTER 2
SYSTEM ANALYSIS

2.1 PROBLEM DEFINITION

Unplanned machinery failures are one of the leading causes of operational disruptions, financial losses, and safety risks in industrial settings. In manufacturing plants, unexpected breakdowns can halt production lines, causing significant revenue losses and delays. In industries like transportation, power generation, and healthcare, system failures can lead to critical failures, life-threatening situations, and infrastructure damage. Equipment degradation happens gradually due to wear and tear, environmental conditions, and operational stress, yet many failures occur suddenly without warning. Traditional maintenance approaches—reactive (fixing after failure) and preventive (routine servicing)—are often inefficient, costly, and fail to detect early failure indicators. Predictive maintenance is an advanced data-driven approach that aims to identify early signs of potential failures before they lead to serious breakdowns. It leverages machine learning, deep learning, IoT sensors, and real-time analytics to monitor equipment health and predict failures.

Sensor-based monitoring systems continuously track parameters such as temperature, vibration, pressure, and electrical signals, allowing AI-driven models to detect anomalies and predict maintenance needs. Research shows that predictive maintenance can reduce equipment downtime by up to 40% and cut maintenance costs by 25%, making it a crucial strategy in Industry 4.0 and smart manufacturing. However, despite its advantages, predictive maintenance still faces challenges in adoption, data security, model reliability, and high implementation costs. Many industries lack the infrastructure to handle large-scale real-time data processing, and AI-based models must be continuously updated to improve accuracy. Additionally, small and medium enterprises (SMEs) often find it costly and complex to integrate predictive maintenance into their systems. By developing a robust AI-driven predictive maintenance system, this project aims to enhance reliability, optimize maintenance schedules, and reduce operational risks. Implementing an automated, data-driven maintenance framework will allow industries to transition from reactive to proactive strategies, improving efficiency, cost-effectiveness, and equipment longevity.

2.2 EXISTING SYSTEM

The existing system for equipment maintenance primarily relies on reactive and preventive maintenance approaches. Reactive maintenance, also known as "run-to-failure," involves repairing machinery only after a failure occurs. While this approach reduces upfront costs, it often leads to unexpected downtime, increased repair costs, and production losses. Preventive maintenance, on the other hand, follows a fixed schedule where equipment is serviced at regular intervals regardless of its actual condition. While it helps in reducing sudden failures, it often results in unnecessary maintenance, increased labor costs, and inefficient resource utilization.

Industries have also implemented condition-based maintenance (CBM), which monitors specific parameters like temperature, vibration, and pressure to determine maintenance needs. However, CBM still lacks predictive capabilities and often fails to detect complex failure patterns. With the rise of Industry 4.0, organizations are gradually integrating Internet of Things (IoT) sensors and basic machine learning models to improve maintenance efficiency. Despite these advancements, most existing systems lack real-time analytics, predictive accuracy, and adaptive learning mechanisms, limiting their ability to accurately forecast failures and optimize maintenance schedules.

DISADVANTAGES

- High Downtime and Unexpected Failures – Reactive maintenance leads to sudden equipment breakdowns, causing production delays, financial losses, and operational risks.
- Inefficient Maintenance Scheduling – Preventive maintenance follows a fixed schedule rather than actual equipment health, leading to unnecessary servicing or unexpected failures.
- Limited Fault Detection Capabilities – Many systems fail to detect early warning signs due to basic threshold-based monitoring, resulting in costly repairs and safety risks.
- High Maintenance Costs – Unplanned repairs require urgent procurement of spare parts and skilled technicians, making maintenance more expensive and resource-intensive.

2.3 PROPOSED SYSTEM

The proposed system introduces a machine learning-based predictive maintenance framework designed to identify early signs of equipment failure and recommend proactive maintenance actions. Unlike traditional maintenance strategies, this system leverages real-time sensor data, intelligent algorithms, and statistical analysis to evaluate the operational condition of machinery and forecast its Remaining Useful Life (RUL). It transitions maintenance operations from time-based and reactive models to a data-driven, predictive approach, enabling timely interventions.

At the core of the system is the integration of supervised and unsupervised machine learning techniques. A Random Forest Regressor is used for predicting the RUL of machines, while a Random Forest Classifier determines whether a machine requires maintenance. For anomaly detection, K-Means Clustering is applied to identify irregular patterns in sensor data, flagging potential failures before they escalate. These models are trained on historical sensor data, including readings such as temperature, vibration, pressure, and current.
The proposed architecture includes components for sensor integration, data preprocessing, model training, and prediction visualization. Raw sensor data undergoes cleaning, normalization, and feature engineering to ensure high-quality inputs for the learning models. The processed data is then fed into the machine learning algorithms to generate outputs indicating machine health and failure probability.

To provide accessibility and interactivity, the system is deployed through a Streamlit-based web dashboard. This interface enables users to input new sensor readings, visualize historical trends, and monitor the real-time status of machinery through color-coded indicators (e.g., Normal, Warning, Critical). The dashboard also supports dynamic prediction outputs, enabling maintenance teams to interpret model results and plan actions accordingly.

ADVANTAGES:

- Reduces Unplanned Downtime – By predicting failures in advance, the system helps schedule maintenance proactively, minimizing sudden equipment breakdowns.
- Optimizes Maintenance Costs – Maintenance is performed only when needed, which avoids unnecessary repairs and reduces labour and spare part expenses.
- Improves Equipment Lifespan – Early detection of issues allows timely action, which helps prevent extensive wear and damage, extending the useful life of machines.
- Increases Operational Efficiency – Real-time monitoring and predictive alerts ensure machines run smoothly, improving productivity and process reliability.
- Enhances Decision Making – Visual insights and machine learning predictions help maintenance teams make informed, data-backed decisions.
- Supports Sustainability – By reducing waste from premature replacements and conserving energy through optimized operation, the system contributes to eco-friendly practices.
- Scalable and Adaptable – The system can be adapted to various types of industrial equipment and scaled to accommodate multiple machines and sensor types.

CHAPTER 3
SYSTEM STUDY

3.1 FEASIBILITY STUDY

The feasibility of the proposed system is evaluated to ensure that it is viable from economic, technical, and performance perspectives. This study helps determine whether the system can be developed within available resources and constraints, while meeting all user requirements. A thorough understanding of the system's key components and environment is essential for a complete feasibility assessment.

3.1.1 Economic Feasibility

Economic feasibility assesses the cost-effectiveness of the system by estimating the expenses involved in its design, development, deployment, and maintenance. Since the proposed system uses freely available open-source technologies such as Python, Streamlit, and Scikit-learn, the cost of development is significantly reduced. Only basic hardware and internet infrastructure are required, which are already available in most industrial settings.

The system demonstrates a high return on investment (ROI) by reducing machine downtime, minimizing maintenance costs, and improving operational efficiency. This makes the project economically justifiable and suitable for industries of varying sizes, including SMEs with limited budgets.

3.1.2 Technical Feasibility

The proposed system is technically feasible as it relies on well-established machine learning models and sensor-based data processing techniques. It integrates Random Forest algorithms for prediction and classification, and K-Means Clustering for anomaly detection—models that are robust, scalable, and compatible with real-time industrial applications.

The software architecture is lightweight and modular, requiring only standard hardware configurations for implementation. Additionally, the use of Streamlit ensures ease of deployment and cross-platform accessibility. Thus, the technical infrastructure required for this system is minimal, making it feasible for real-world application without the need for significant changes or upgrades to the existing setup.

3.1.3 Performance Feasibility

Performance feasibility evaluates whether the system will operate efficiently and meet expected performance standards under real-time industrial conditions. The predictive models used in the system have been trained and validated on real-world datasets, ensuring high accuracy and reliability in identifying faults, estimating remaining useful life, and detecting anomalies. The system supports real-time data visualization, quick prediction outputs, and efficient decision-making, ensuring timely maintenance interventions. The use of a responsive and lightweight web dashboard guarantees a smooth user experience even on low-end systems.

The models are optimized for minimal latency and fast response, which is critical in industrial environments where immediate insights are necessary to prevent failures. Additionally, the system can scale to handle multiple machines and increasing data volumes without compromising on speed. The predictive framework is designed to adapt and retrain on new data, improving accuracy over time. By ensuring consistent performance under varying conditions, the system proves to be dependable for continuous monitoring. Overall, it meets the speed, accuracy, and usability standards required for modern industrial applications.

CHAPTER 4
SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

Component	Specification
Processor	Intel Core i5 or higher (3.0 GHz or above)
RAM	Minimum 4 GB
Storage	512 GB
Graphics	Integrated GPU
Display	15-inch Monitor or larger
Network	Stable internet connection (for data access and updates)

4.2 SOFTWARE REQUIREMENTS

Software	Specification
Operating System	Windows 10 / 11, Linux (Ubuntu), or macOS
Programming Language	Python 3.8 or above
IDE	Jupyter Notebook, Visual Studio Code
Dashboard Framework	Streamlit (for web-based interface)
Version Control	Git & GitHub
Browser	Chrome / Firefox (for dashboard visualization)

4.3 SOFTWARE DESCRIPTION:

Operating System : The system is compatible with Windows 10/11, Linux (Ubuntu), and macOS operating systems. These platforms provide robust support for Python environments, machine learning libraries, and data processing tools. Linux is particularly popular for machine learning due to its performance and ease of environment configuration, while Windows and macOS are user-friendly and widely used for development and testing. The system is platform-independent and can be deployed across any of these environments with minimal configuration.

Python 3.8 or above : Python is the core programming language used for building this predictive maintenance system due to its simplicity, versatility, and vast ecosystem of data science libraries. Version 3.8 or above is recommended to ensure compatibility with modern packages such as pandas, scikit-learn, seaborn, NumPy, and Streamlit. Python supports object-oriented programming, making it ideal for building modular, scalable systems like predictive maintenance applications.

Pandas : pandas is a powerful data manipulation and analysis library. It is used for loading datasets, handling missing values, transforming sensor data, and organizing time-series records. DataFrames in pandas allow for easy indexing, filtering, grouping, and merging of large datasets typically used in predictive maintenance.

NumPy : NumPy supports numerical operations on large, multi-dimensional arrays and matrices. It is used extensively for mathematical computations, generating numerical features, scaling, and converting sensor data into model-ready formats. It also powers many functions inside pandas, scikit-learn, and other ML libraries.

Scikit-learn : scikit-learn is the primary machine learning library used for model development. It provides:

RandomForestRegressor for predicting Remaining Useful Life (RUL), RandomForestClassifier for maintenance status prediction, KMeans for anomaly detection . It also supports model evaluation, cross-validation, hyperparameter tuning, and feature selection.

Matplotlib & Seaborn : These are visualization libraries used to plot sensor trends, distribution graphs, and correlation heatmaps. matplotlib provides low-level plotting, while seaborn builds on it to generate attractive statistical visuals. They are useful for EDA (Exploratory Data Analysis) and report generation.

joblib / pickle : These libraries are used for saving (serializing) and loading trained machine learning models. Once a model is trained, it can be stored as a .pkl or .joblib file and reused for prediction without retraining.

Dashboard Framework : Streamlit is a fast, open-source Python framework used to build interactive web applications specifically for machine learning and data science projects. Unlike traditional frameworks like Flask or Django, Streamlit requires no front-end coding or web development knowledge. It allows developers to transform data scripts into functional dashboards with just a few lines of Python code. This makes it an ideal tool for quick prototyping, experimentation, and showcasing data-driven models in a visually appealing format.

Streamlit supports real-time interaction through widgets such as sliders, buttons, and file uploaders, enabling users to provide inputs and receive instant feedback from models. It integrates smoothly with popular Python visualization libraries like matplotlib, seaborn, and plotly, making it easy to create dynamic and insightful visual outputs. Trained machine learning models can be deployed within the app for live prediction. Moreover, Streamlit is lightweight and responsive, and applications can be run locally or hosted on platforms like Heroku, AWS, or Streamlit Cloud for broader accessibility and deployment.

Browser – Chrome / Firefox : The deployed Streamlit application can be accessed using any modern web browser, with Google Chrome and Mozilla Firefox being the most recommended due to their fast performance and compatibility with JavaScript-based visualization elements. These browsers support the rendering of interactive charts, color-coded indicators, and form components that are essential for user interaction with the predictive maintenance dashboard. A stable browser ensures smooth performance and a seamless user experience.

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

It begins with collecting raw sensor data from various sources, which then undergoes preprocessing to remove noise and prepare meaningful features. This cleaned data is used for training two machine learning models: Random Forest and K-Means Clustering. Random Forest helps in estimating the Remaining Useful Life (RUL) and maintenance needs of machinery. K-Means Clustering is utilized to detect anomalies that may indicate early signs of failure.

The processed results are delivered through a user-friendly web application. Users can input real-time sensor readings and instantly receive outputs such as maintenance status, RUL, and anomaly alerts. Visual indicators like color-coded statuses enhance quick understanding and decision-making. The system ensures accurate predictions and real-time updates to prevent unexpected downtimes. Ultimately, it supports proactive maintenance, reducing costs and improving equipment efficiency.

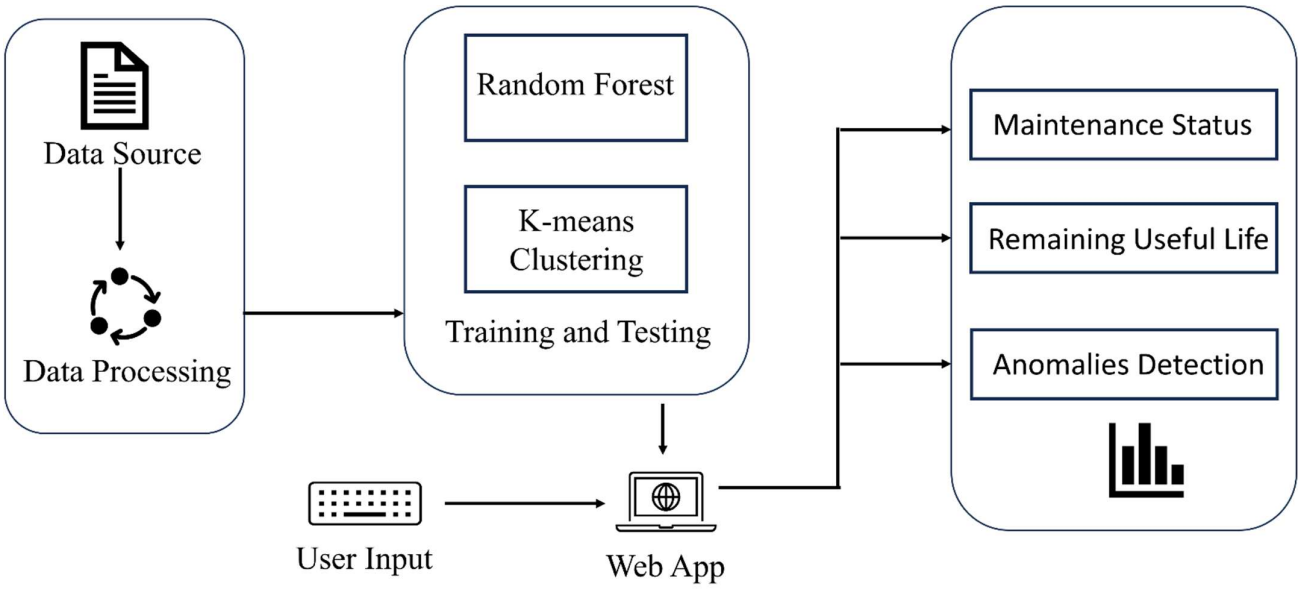


Fig 5.1 System Architecture

5.2 CLASS DIAGRAM

The class diagram outlines key components of a predictive maintenance system. The Dataset class holds core attributes like sensor_data, timestamp, and failure_label, which are essential for model training. The Data preprocessing class offers methods to clean and manage missing values, ensuring quality input data. The Model training class contains functions for training machine learning models like Random Forest and KMeans. It also includes an evaluate_model() method to assess performance.

The Prediction class performs crucial tasks like predicting RUL (Remaining Useful Life), classifying maintenance status, and detecting anomalies. The Web Application class enables user interaction, offering functionalities such as uploading data, displaying predictions, and visualizing insights. This diagram clearly separates data handling, model development, and user interface. It promotes modularity and easy maintenance of the system. Overall, the structure supports a robust and scalable predictive maintenance workflow.

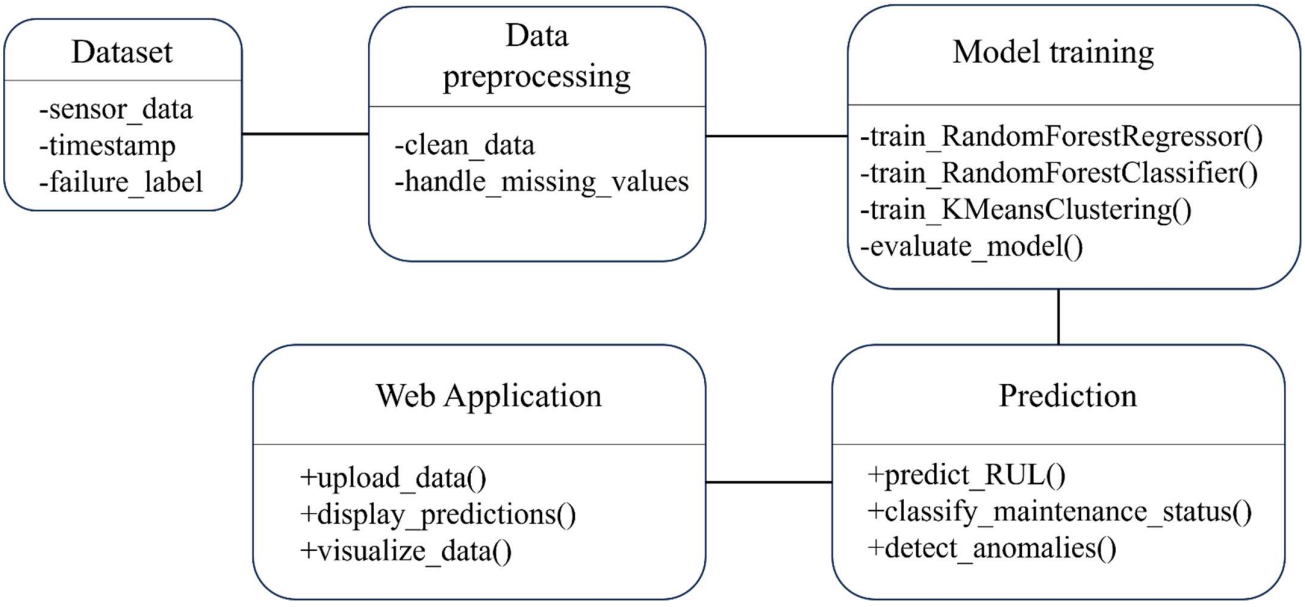


Fig 5.2 Class Diagram

5.3 USE CASE DIAGRAM

The use case diagram represents the interaction between users and the predictive maintenance system. The user initiates the process by uploading sensor data, which serves as the foundation for all subsequent operations. The system processes this data to predict the Remaining Useful Life (RUL) of the equipment. Based on this prediction, the system updates the maintenance status, indicating whether attention is required. This proactive approach enables timely maintenance planning and resource allocation.

In addition to RUL prediction and maintenance updates, the system can detect anomalies in real-time, identifying abnormal sensor patterns that could indicate future failures. These predictions and diagnostics are then presented visually, offering clear insights for engineers or maintenance personnel. The use case diagram emphasizes key functionalities such as data input, analysis, monitoring, and visualization. This allows for effective communication between the user and the system. Ultimately, the diagram outlines a structured workflow that enhances operational efficiency.

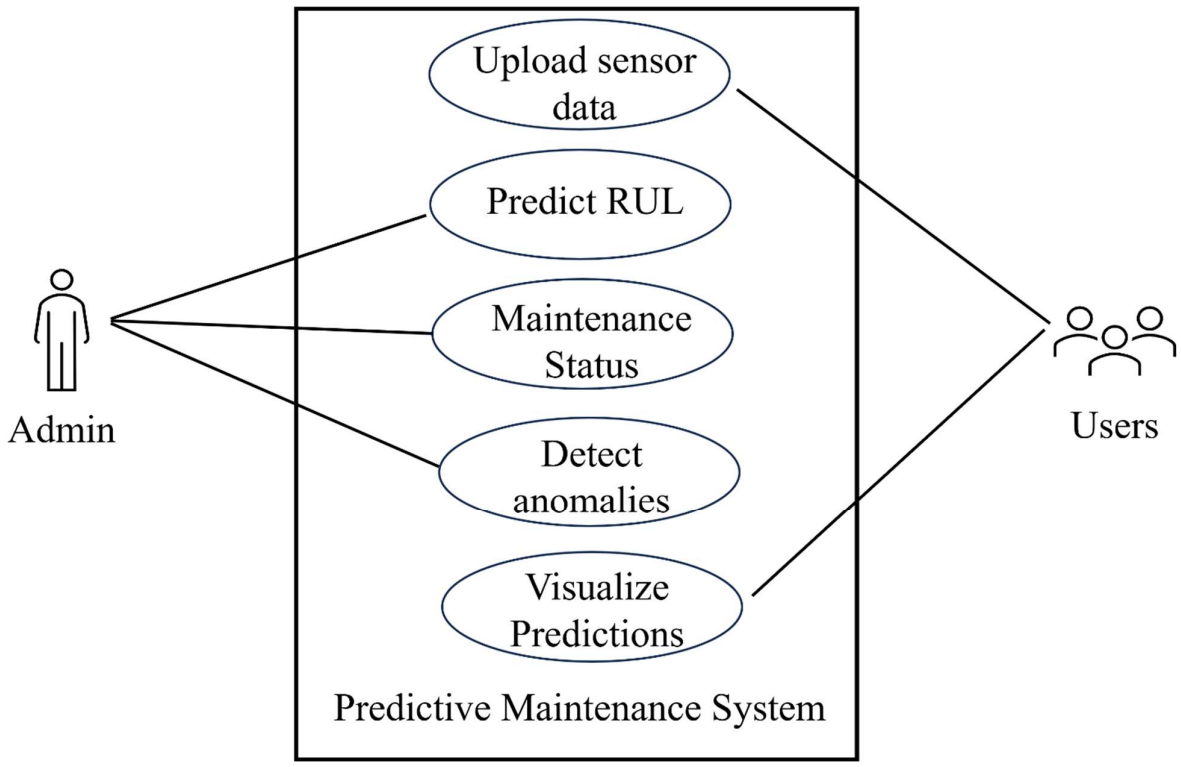


Fig 5.3 Use Case diagram

CHAPTER 6

PROPOSED ALGORITHM IMPLEMENTATION

6.1 PROJECT DESCRIPTION

Predictive Maintenance is an intelligent system designed to predict machine failures and maintenance requirements before they occur, thereby reducing downtime and operational costs. This project focuses on building a machine learning-based system that can analyze sensor data, detect anomalies, predict the Remaining Useful Life (RUL) of equipment, and classify the maintenance status of industrial components. By leveraging advanced analytics and visualization, this system supports decision-making in industrial environments and improves overall equipment effectiveness (OEE).

The system architecture is divided into multiple components, each handling a specific task in the workflow. The process begins with uploading time-series sensor data from machines, which includes various features such as temperature, pressure, and vibration. The uploaded data undergoes preprocessing to remove noise and handle missing values, ensuring the dataset is clean and consistent. This stage is crucial for accurate model training and predictions. Data preprocessing enhances the reliability of the system and prepares it for robust machine learning analysis.

Once the data is preprocessed, it is passed into the model training phase. Multiple models are trained using Random Forest Regressor, Random Forest Classifier, and KMeans Clustering algorithms. The regressor is used for predicting the Remaining Useful Life (RUL), the classifier for maintenance status classification, and clustering helps in anomaly detection. After training, the models are evaluated using metrics like accuracy, F1 score, and mean squared error. This ensures that only high-performing models are used for real-time predictions in the production environment.

The prediction module uses the trained models to forecast future maintenance needs. It includes methods like predict_RUL(), classify_maintenance_status(), and detect_anomalies(). These predictions are sent to the front-end Web Application, which provides a user-friendly interface for data upload, prediction display, and visualization. Users can interact with the system easily, understand the health status of equipment through visual reports, and take preventive actions in advance. The system thus combines powerful backend logic with a responsive front-end interface.

This project finds applications in manufacturing plants, smart factories, transportation fleets, and energy systems where predictive maintenance is critical. By automating maintenance insights, this system not only minimizes unplanned downtime but also extends the lifespan of machines. With continuous improvements and integration of real-time data streams, the Predictive Maintenance System offers scalable and intelligent support for modern industrial operations.

In addition to its functional capabilities, the system was designed with modularity and scalability in mind, allowing easy integration with IoT platforms and cloud services. Security and data integrity are also considered, ensuring that sensitive operational data remains protected. The web application is built using modern web technologies and allows real-time monitoring and alert generation. Furthermore, model retraining mechanisms are integrated to ensure that the predictions stay accurate as more data is collected. The entire solution is ideal for both batch and live processing modes, offering flexibility in deployment. This makes the system well-suited for adoption in Industry 4.0 environments, where automation, analytics, and machine intelligence drive smarter decision-making.

6.2 MODULES

- Dataset
- Data Preprocessing
- Training and Evaluation
- Prediction
- Visualisation

6.3 MODULE DESCRIPTION

6.3.1 Dataset

The dataset used in this project contains time-series data collected from industrial equipment during regular operations. Each record captures readings from multiple sensors such as sensor_1, sensor_2, and sensor_3. These values represent physical conditions like temperature, vibration, and pressure. Additional fields include operation_cycle, maintenance_flag, and RUL (Remaining Useful Life). The maintenance_flag indicates whether maintenance was required at that cycle. RUL acts as the target variable for predictive modeling. The data supports both supervised and unsupervised learning approaches. It covers various operating conditions and potential failure

scenarios. This allows for training models that generalize well across different machine behaviors. The dataset structure also supports real-time and batch processing use cases.

Preprocessing is essential to make the data suitable for machine learning. It involves handling missing values, smoothing fluctuations, and applying normalization. Feature engineering adds new insights through rolling statistics and trend detection. Sensor patterns over time are analyzed to predict failure points and maintenance needs. Clustering techniques help in identifying abnormal behaviors without labeled data. Supervised models like Random Forest are trained to estimate RUL and classify maintenance conditions. The dataset's balance and volume make it suitable for scalable deployment. Its temporal nature allows the models to adapt to evolving equipment conditions. Overall, the dataset forms the core for building an intelligent predictive maintenance solution.

	A	B	C	D	E	F
1	sensor_1	sensor_2	sensor_3	operation	maintenar	RUL
2	0.49671	1.39936	-0.67518	831	0	4167.78
3	-0.13826	0.92463	-0.14452	4061	0	937.137
4	0.64769	0.05963	-0.79242	4801	0	197.222
5	1.52303	-0.64694	-0.30796	2604	0	2393.65
6	-0.23415	0.69822	-1.89361	714	0	4285.08
7	-0.23414	0.39349	0.21329	3906	0	1092.71
8	1.57921	0.89519	0.00121	1819	0	3177.24
9	0.76743	0.63517	-0.81709	4869	0	126.65
10	-0.46947	1.04955	0.65925	1687	0	3307.41
11	0.54256	-0.53524	0.93757	1711	0	3282.47
12	-0.46342	1.31739	-1.60756	996	0	3998.22
13	-0.46573	0.1976	-0.76272	377	0	4618.25
14	0.24196	2.07526	-0.76914	4747	0	246.702
15	-1.91328	-0.68919	-0.9399	1736	0	3261.24
16	-1.72492	1.73596	0.82947	2258	0	2738.4
17	-0.56229	0.19791	-0.19383	2532	0	2464.96
18	-1.01283	-0.65142	-0.26451	4004	1	994.891
19	0.31425	-0.48389	-2.00386	4956	0	45.0645
20	-0.90802	-0.32035	0.63542	3362	0	1639.66
21	-1.4123	0.42417	-1.23926	1549	0	3454.88
22	1.46565	0.52284	0.05993	4600	0	401.836
23	-0.22578	-0.5737	0.27738	1332	0	3670.36
24	0.06753	-0.02435	1.36066	4217	0	783.955
25	-1.42475	2.14227	-1.30882	777	0	4224.55
26	-0.54438	1.72754	-3.01951	2758	0	2245.38
27	0.11092	0.43632	0.18385	2128	0	2874.65
28	-1.15099	0.038	1.80051	3858	0	1143.96

Fig 6.1 Sample Dataset

6.3.2 Data Preprocessing

Data preprocessing is a critical step in the development of any machine learning model, particularly in predictive maintenance systems, where sensor data is collected continuously and often exhibits inconsistencies. Raw industrial data typically contains missing values, outliers, noise, and irrelevant features, all of which can negatively impact model performance if not properly handled. The primary goal of preprocessing is to prepare the dataset into a clean and structured format suitable for training accurate and efficient predictive models.

The dataset used in this project includes sensor readings, operational cycle information, and maintenance labels. The preprocessing pipeline begins with the inspection and analysis of the dataset's completeness. Missing values are treated through imputation techniques such as mean substitution or forward-fill based on temporal trends. In this project, rows with missing values were carefully analyzed and either imputed or removed depending on their frequency and impact. Duplicate records, if present, were also eliminated to ensure data integrity.

Once the data is cleaned, normalization is applied to the sensor readings to ensure all features contribute equally to the model. Sensor values have different ranges, and normalization scales them to a common range, typically between 0 and 1, or standardized with a mean of 0 and standard deviation of 1. This step is particularly important when using distance-based algorithms or neural networks, which are sensitive to feature scale.

Outlier detection is another essential task in the preprocessing phase. In industrial equipment, extreme sensor values may indicate real-world anomalies or sensor malfunctions. Using statistical techniques like Z-score or IQR (Interquartile Range), outliers were identified. Depending on context, some were retained for training anomaly detection models, while others, confirmed to be noise, were excluded. Visualization tools like box plots and time-series graphs were used to support the decision-making process in handling these outliers.

Lastly, the dataset was split into training and testing sets. A proper split was used to simulate real-world deployment, where past data is used to predict future failures. Cross-validation techniques were applied to evaluate the consistency and generalizability of the model.

6.3.3 Training and Evaluation

The training and evaluation phase forms the core of the predictive maintenance pipeline, where machine learning models are developed, trained on historical data, and assessed for their ability to generalize to unseen equipment behavior. For this project, separate models were trained for different tasks: Remaining Useful Life (RUL) prediction, maintenance classification, and anomaly detection using unsupervised clustering.

For RUL prediction, a regression model was trained using sensor readings and operation cycle data as input features. The target variable was the numerical RUL value. The dataset was first normalized to ensure consistent scale across features, and rolling statistical features were engineered to capture trends in sensor values. A Random Forest Regressor was selected due to its robustness to noisy features and ability to handle non-linear relationships. The model was trained using a time-based split to simulate real-world deployment—training on earlier machine cycles and testing on later ones. Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) were used as evaluation metrics, as they provide insight into the model's average and penalized errors respectively.

For the maintenance status classification task, a binary classifier was trained where the maintenance flag served as the label. The same set of input features was used, and a Random Forest Classifier was employed for its interpretability and high accuracy. The model was trained using a stratified split to maintain class balance, and evaluated using standard classification metrics including Accuracy, Precision, Recall, and F1 Score. The confusion matrix was also analyzed to detect any misclassification trends, particularly false negatives, which could result in unplanned equipment failure.

An unsupervised K-Means clustering approach was used for anomaly detection. The model was trained only on normal operating conditions (maintenance flag = 0), and the clustering algorithm was used to detect deviation from normal patterns in high-dimensional sensor data. The optimal number of clusters (k) was determined using the Elbow Method. Distance-based anomaly scores were computed for each observation, and those with high scores were flagged as anomalies. These flagged points were cross-verified against actual maintenance events for validation. The clustering results were visualized using PCA and t-SNE for dimensionality reduction to interpret cluster separation.

All models were validated using cross-validation where applicable, and hyperparameters were tuned using Grid Search to optimize model performance. During evaluation, care was taken to avoid data leakage by ensuring no future information was used during training. The models were

also tested on edge cases—such as machines with extremely short or long lifespans—to evaluate robustness.

The outcome of the training and evaluation process demonstrated the effectiveness of combining regression, classification, and clustering for predictive maintenance. The regression model achieved low prediction errors in estimating RUL, the classifier accurately identified maintenance needs, and the clustering model effectively flagged operational anomalies. Together, these models contribute to a reliable and intelligent system for industrial equipment health monitoring.

6.3.4 Prediction

RUL(Remaining Useful Life) : The RUL prediction module estimates how many operational cycles remain before a machine is likely to fail. Using a trained Random Forest Regressor, the model analyzes real-time sensor data and historical degradation patterns to calculate a numerical RUL value. This insight helps maintenance teams prioritize servicing and optimize machine usage, reducing downtime and cost. RUL values are updated with each new cycle and visualized through trend graphs for intuitive interpretation.

Anomaly Detection : Anomaly detection identifies unusual sensor behavior that may indicate early-stage faults. K-Means clustering is used to group normal operational patterns and flag deviations as anomalies. These outliers often represent abnormal wear, overheating, or mechanical instability. Alerts are triggered when a data point falls outside its expected cluster. Visual tools like scatter plots and cluster maps help operators understand sensor deviations and respond proactively.

Maintenance Status Classification : This model predicts whether maintenance is immediately required based on current sensor readings. A Random Forest Classifier evaluates time-series inputs and outputs a binary label — “Maintenance Required” or “Normal.” It considers trends across multiple features and provides fast, real-time decisions. This prediction helps avoid reactive maintenance and supports smart scheduling. Maintenance flags are color-coded in dashboards to simplify monitoring.

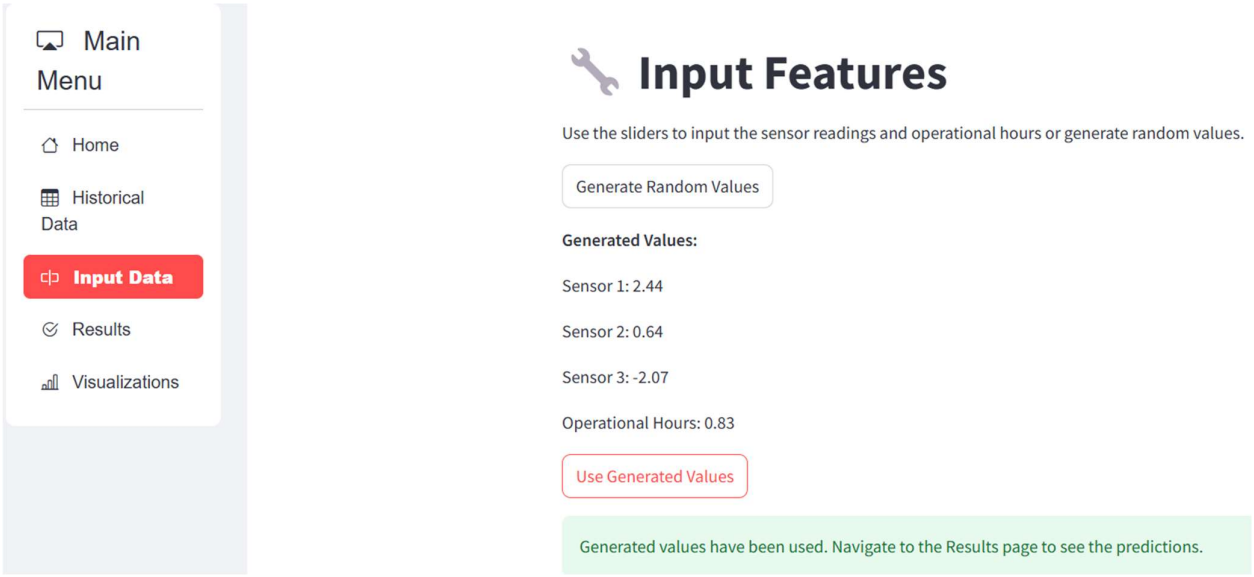


Fig 6.2 Input for Prediction

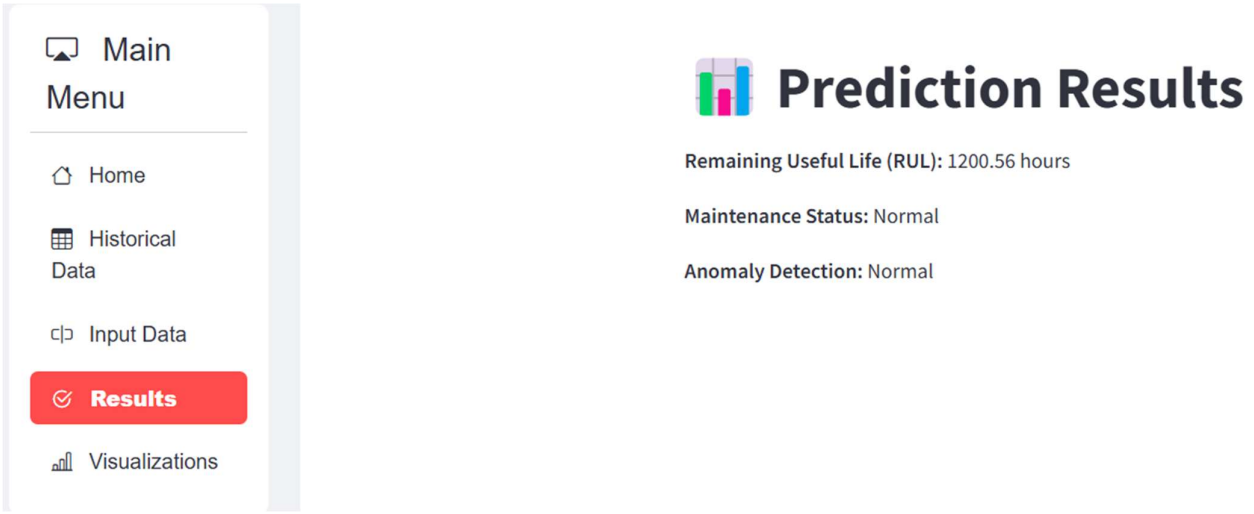


Fig 6.3 Prediction

6.3.5 Visualisation

Histogram of Sensor Readings : Histograms display the frequency distribution of generated sensor_1, sensor_2, and sensor_3 values. They help identify normal operating ranges and sensor anomalies. Frequent values reflect stable machine behavior, while rare spikes indicate potential issues. This aids in feature selection and pattern recognition. The plots update dynamically with generated inputs.

Scatter Plot of Sensor Readings vs Operational Hours : This plot shows the correlation between sensor readings and operational cycles. It reveals how sensor behavior changes with machine usage. Patterns in the scatter plot help detect degradation trends and outliers. Clusters indicate stable operation, while isolated points suggest anomalies. Generated input simulates real-time machine behavior.

Line Chart of RUL Over Time : The RUL line chart tracks the predicted lifespan of equipment across cycles. It shows how RUL decreases with continuous use. Sudden drops in the line indicate accelerated degradation. The chart is updated in real-time with each prediction. It helps maintenance teams plan timely interventions.

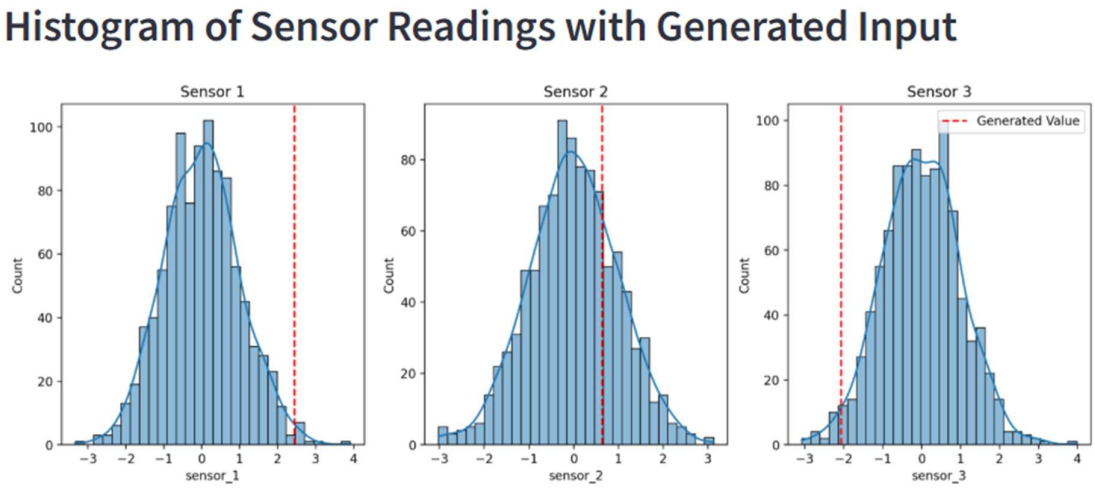


Fig 6.4 Chart for Sensor Readings

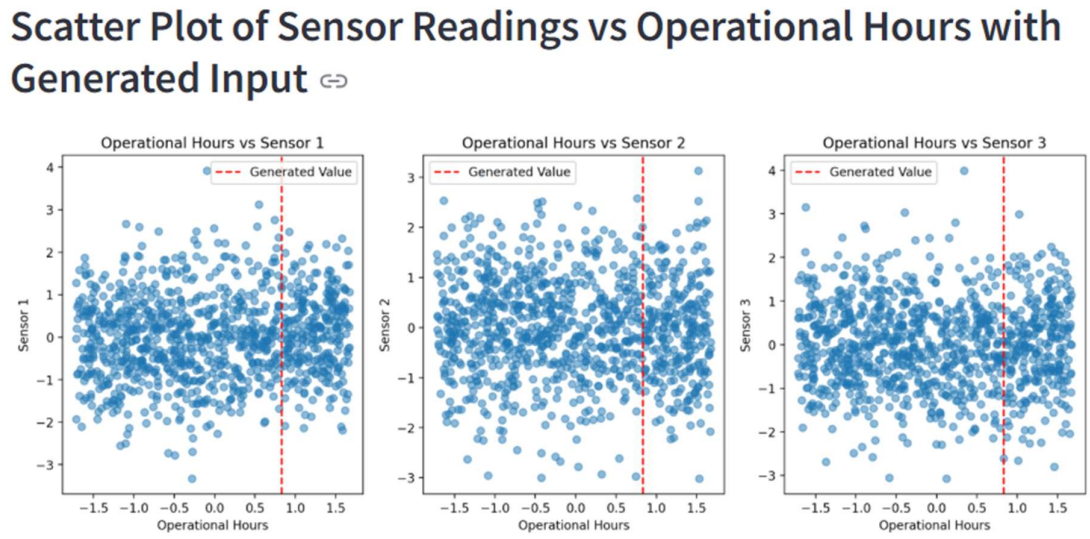


Fig 6.5 Chart for Sensor Vs Operational Hours

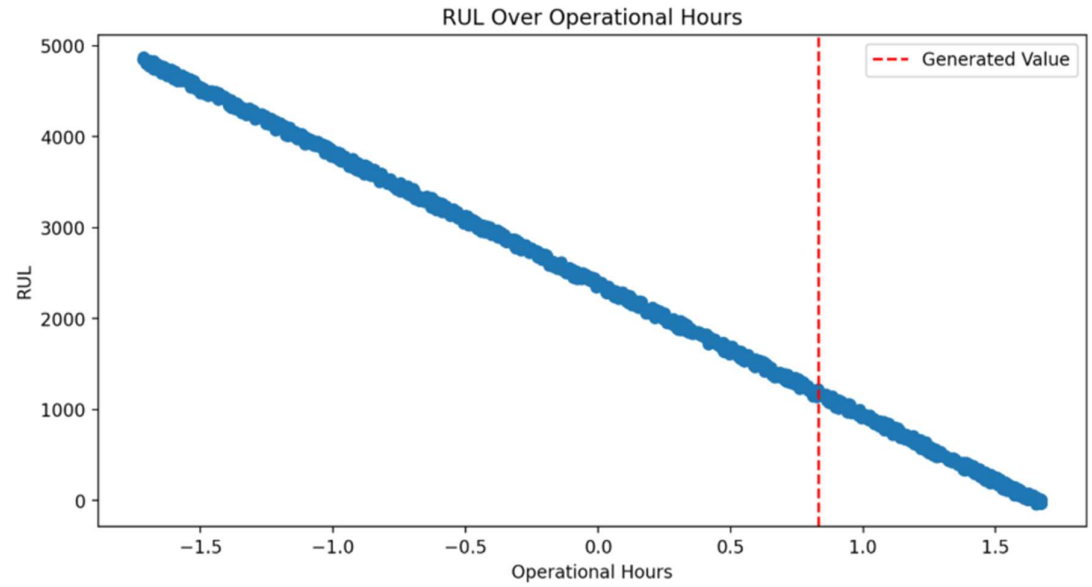


Fig 6.6 RUL Visualisation

CHAPTER 7
RESULTS AND DISCUSSION

The predictive models developed in this project have demonstrated strong performance in simulating real-world industrial maintenance scenarios. The Random Forest Regressor used for Remaining Useful Life (RUL) prediction shows its ability to accurately estimate machine degradation over time. The model effectively recognized time-dependent sensor trends and provided early warnings about component wear, which is essential for minimizing unplanned downtimes in industrial systems.

The Random Forest Classifier for maintenance status performed with high precision and recall, correctly identifying components that required immediate servicing. This model maintained balanced accuracy even with varying sensor behaviors and handled imbalanced datasets using stratified sampling. The confusion matrix confirmed that false negatives—critical in maintenance planning—were minimal. This classification layer supports industries in avoiding both premature replacements and reactive breakdown handling.

An unsupervised K-Means clustering model was employed for anomaly detection, allowing the system to detect abnormal patterns without prior labels. It effectively separated normal operating behavior from outliers such as abnormal vibration or temperature spikes, which often precede failure events. These results were validated through visualizations, where cluster separation clearly highlighted potential risks.

Visualization tools including line charts, histograms, and scatter plots helped convey predictive insights in a user-friendly manner. These tools made it easier for maintenance teams to interpret predictions, monitor real-time machine conditions, and take preventive actions. Overall, the results validate the practicality of this multi-model approach, which can be seamlessly integrated into industrial predictive maintenance workflows to reduce costs, improve asset utilization, and increase operational uptime.

CHAPTER 8
SYSTEM TESTING

8.1 TESTING

Software testing is an essential phase in the development of a machine learning-based system to ensure that each component functions correctly and as expected under real-world conditions. For the Predictive Maintenance system, testing was conducted at different levels: Unit Testing, Integration Testing, and Validation Testing. These layers of testing help in identifying bugs early, verifying inter-module consistency, and ensuring that the overall system meets the desired objectives of prediction accuracy and reliability.

8.2 UNIT TESTING

Unit testing focuses on verifying the functionality of individual components or modules in isolation. In this project, unit tests were applied to key components such as the data preprocessing pipeline, RUL prediction model, anomaly detection logic, and sensor input simulation module. Each function, including feature scaling, outlier handling, and prediction outputs, was tested independently using mock data. Python-based test scripts were used to validate function outputs against expected values. For example, sensor normalization functions were tested to ensure correct scaling across different sensor ranges. These tests helped detect minor logic errors and ensured the reliability of each component before integration.

8.3 INTEGRATION TESTING

Integration testing was performed to ensure that different modules of the system — including data loading, preprocessing, model inference, and visualization — work together seamlessly. The end-to-end flow from generated sensor input to prediction and dashboard display was tested using synthetic input values. Particular attention was given to the consistency of data passed between modules, such as correct formatting of preprocessed features being fed into the Random Forest model. Integration testing validated the interaction between RUL regression, maintenance classification, and clustering models, ensuring that all outputs were correctly synchronized for display and alerts. Any mismatches in data shapes or parameter passing were resolved during this phase.

8.4 VALIDATION TESTING

Validation testing was conducted to ensure that the final system meets user and project requirements. This involved comparing the model predictions and system outputs against expected outcomes using test datasets. The accuracy of the RUL predictions was validated using known degradation patterns, while maintenance status classification and anomaly detection results were cross-verified with labeled inputs. The visualization dashboard was also tested to confirm real-time updating and intuitive presentation. Validation confirmed that the system achieved its objectives: timely predictions, anomaly alerts, and actionable insights for proactive maintenance planning.

8.5 TESTING REPORT

Test Case id and Name	Test Inputs	Actions	Expected Results	Test Case Pass Criteria	Test Case Fail Criteria	Achieved
#01 Generate Input	>Random Sensor Data	Randomly assigns value to sensor for prediction	Random Values of 3 sensors need to be displayed	The test case passes if Random values are fetched	The test case fails if random values are not displayed	Pass
#02 Manual Input	>Sensor Data	Reading of sensor data is fed for prediction	Real data need to be submitted without error	The test case passes if Non-zero data is given	The test case fails if all sensor has zero as value	Pass
#03 Prediction of result	>OnClickSubmit	Returns the RUL, Maintenance Status	Prediction should be displayed with 3 parameters	The test case passes if model prediction function works and returns the result	The test case fails if model prediction function does not work	Pass

Table 8.1 Test Cases

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

The predictive maintenance system developed in this project successfully applies machine learning techniques to monitor and analyze industrial equipment health. By integrating RUL estimation, anomaly detection, and maintenance classification, the system enables timely interventions and reduces unplanned downtime. The use of Random Forest and K-Means models ensures reliable predictions and effective decision-making support. Visual dashboards and trend graphs offer intuitive insights for operators and maintenance teams. This approach enhances operational efficiency and supports data-driven maintenance planning.

The system architecture is modular, making it adaptable across various equipment types and manufacturing setups. Real-time visualization of predictions and sensor trends ensures high transparency in machine condition monitoring. Testing confirmed that the system maintains accuracy and stability under different operational scenarios. With its practical design and accurate output, the solution demonstrates the value of AI-driven predictive maintenance. It reflects a strong step toward smarter and more efficient industrial maintenance practices.

9.2 FUTURE ENHANCEMENT

- Integration with IoT and Edge Devices : Incorporate real-time data streaming from IoT-enabled industrial sensors to enhance live monitoring and reduce latency in prediction pipelines. Edge computing can be used to process data locally for faster anomaly detection.
- Deep Learning Models for Time-Series Forecasting : Implement advanced models like LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Units) to improve RUL prediction by learning complex temporal dependencies in sequential sensor data.
- Automated Model Retraining with Continuous Learning : Enable online learning mechanisms that allow models to retrain periodically with newly collected data, ensuring that predictions remain accurate over time as machines undergo wear or environmental changes.

- Cloud-Based Predictive Maintenance Platform : Deploy the system on cloud platforms such as AWS, Azure, or GCP to facilitate remote access, centralized monitoring of multiple machines, and scalable infrastructure for large-scale industrial environments.
- Integration with CMMS (Computerized Maintenance Management Systems): Link prediction outputs with existing enterprise maintenance systems to automate work order generation and maintenance scheduling based on predicted failures.
- Enhanced Fault Diagnosis with Multi-Label Classification : Extend the classification system to not just detect a need for maintenance, but also classify the type of fault (e.g., overheating, imbalance, lubrication issue), improving diagnostic accuracy.
- Real-Time Alert System with Multi-Channel Notifications : Implement real-time alert mechanisms via SMS, email, or mobile app notifications to inform maintenance teams immediately when anomalies or low RUL thresholds are detected.
- Visual Analytics Dashboard with Predictive KPIs : Improve the dashboard with customizable visualizations and KPIs like MTBF (Mean Time Between Failures), MTTR (Mean Time to Repair), and cost-saving projections from predictive scheduling.
- Cross-Equipment Model Generalization : Train generalized models that can adapt to different machine types and configurations with minimal retraining, enabling deployment across diverse industrial settings.
- Integration of Environmental and External Factors : Incorporate additional contextual data such as ambient temperature, humidity, or machine load, which may influence wear patterns and improve prediction robustness.

APPENDIX

A.1 SOURCE CODE

```
import pandas as pd

import numpy as np

# Generating mock data

np.random.seed(42)

data = pd.DataFrame({

    'sensor_1': np.random.normal(0, 1, 1000),

    'sensor_2': np.random.normal(0, 1, 1000),

    'sensor_3': np.random.normal(0, 1, 1000),

    'operational_hours': np.random.randint(100, 5000, 1000),

    'maintenance': np.random.choice([0, 1], 1000, p=[0.95, 0.05])

})

# Simulating remaining useful life (RUL) based on operational hours and sensor readings

data['RUL'] = 5000 - data['operational_hours'] - (data['sensor_1'] + data['sensor_2'] + data['sensor_3']).cumsum()

# Save to CSV

data.to_csv('machinery_data.csv', index=False)


from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

# Load data

data = pd.read_csv('machinery_data.csv')
```

```
# Handle missing values if any

data.fillna(method='ffill', inplace=True)

# Feature selection

features = ['sensor_1', 'sensor_2', 'sensor_3', 'operational_hours']

target_rul = 'RUL'

target_maintenance = 'maintenance'

# Normalize features

scaler = StandardScaler()

data[features] = scaler.fit_transform(data[features])
```

Split data for regression and classification

```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(data[features], data[target_rul], test_size=0.2, random_state=42)

X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(data[features], data[target_maintenance], test_size=0.2, random_state=42)


from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error

# Train regression model

reg_model = RandomForestRegressor(n_estimators=100, random_state=42)

reg_model.fit(X_train_reg, y_train_reg)
```

Predict and evaluate

```
y_pred_reg = reg_model.predict(X_test_reg)

mse_reg = mean_squared_error(y_test_reg, y_pred_reg)

print(f"Regression Model MSE: {mse_reg}")
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
# Train classification model
clf_model = RandomForestClassifier(n_estimators=100, random_state=42)
clf_model.fit(X_train_clf, y_train_clf)
# Predict and evaluate
y_pred_clf = clf_model.predict(X_test_clf)
accuracy_clf = accuracy_score(y_test_clf, y_pred_clf)
print(f"Classification Model Accuracy: {accuracy_clf}")
print(classification_report(y_test_clf, y_pred_clf))

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
# Train clustering model
kmeans = KMeans(n_clusters=2, random_state=42)
data['cluster'] = kmeans.fit_predict(data[features])
# Visualize clustering results
plt.scatter(data['sensor_1'], data['sensor_2'], c=data['cluster'])
plt.xlabel('Sensor 1')
plt.ylabel('Sensor 2')
plt.title('Clustering of Sensor Data')
plt.show()

def predict_maintenance(features):
```

```
    # Predict RUL
    rul_pred = reg_model.predict([features])
    # Predict maintenance status
    maint_pred = clf_model.predict([features])
    # Predict cluster (anomaly detection)
    cluster_pred = kmeans.predict([features])
    return {
        'RUL Prediction': rul_pred[0],
        'Maintenance Prediction': 'Needs Maintenance' if maint_pred[0] == 1 else 'Normal',
        'Anomaly Detection': 'Anomaly' if cluster_pred[0] == 1 else 'Normal'
    }
# Example prediction
sample_features = [0.1, 0.2, 0.3, 1000]
prediction = predict_maintenance(sample_features)
print(prediction)


import streamlit as st
from streamlit_option_menu import option_menu
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
```

```
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
# Prediction function
def predict_maintenance(features):
    rul_pred = reg_model.predict([features])
    maint_pred = clf_model.predict([features])
    cluster_pred = kmeans.predict([features])
    return {
        'RUL Prediction': rul_pred[0],
        'Maintenance Prediction': 'Needs Maintenance' if maint_pred[0] == 1 else 'Normal',
        'Anomaly Detection': 'Anomaly' if cluster_pred[0] == 1 else 'Normal'
    }
# Streamlit Option Menu
with st.sidebar:
    selected = option_menu(
        menu_title="Main Menu",
        options=["Home", "Historical Data", "Input Data", "Results", "Visualizations"],
        icons=["house", "table", "input-cursor", "check2-circle", "bar-chart-line"],
        menu_icon="cast",
        default_index=0,
    )
if selected == "Home":
    st.title("Welcome to the Predictive Maintenance Dashboard")
    st.markdown("""
```

```
    <div style="font-size: 24px;">
        This application provides predictive maintenance insights for industrial machinery.<br><br>
        Use the navigation menu to explore different sections of the app:
        <ul>
            <li>Historical Data - View past machinery data</li>
            <li>Input Data - Enter or generate new sensor readings</li>
            <li>Results - See maintenance predictions</li>
            <li>Visualizations - Analyze data through graphs</li>
        </ul>
    </div>
    """, unsafe_allow_html=True)
elif selected == "Historical Data":
    st.title("📂 Historical Data")
    st.markdown("""
        <style>
        .stDataFrame {
            font-size: 20px;
        }
        </style>
    """, unsafe_allow_html=True)
    st.dataframe(data.head(10).style.set_properties(**{
        'font-size': '20px',
        'padding': '10px'
    }))
```

```
elif selected == "Input Data":
    st.title("🔧 Input Features")
     st.markdown("Use the sliders to input the sensor readings and operational hours or generate random values.")
    if 'generated_values' not in st.session_state:
        st.session_state['generated_values'] = None
    if st.button('Generate Random Values'):
        sensor_1 = np.random.uniform(data['sensor_1'].min(), data['sensor_1'].max())
        sensor_2 = np.random.uniform(data['sensor_2'].min(), data['sensor_2'].max())
        sensor_3 = np.random.uniform(data['sensor_3'].min(), data['sensor_3'].max())
                    operational_hours = np.random.uniform(data['operational_hours'].min(), data['operational_hours'].max())
        st.session_state['generated_values'] = [sensor_1, sensor_2, sensor_3, operational_hours]
        st.success("Random values generated successfully!")
    if st.session_state['generated_values'] is not None:
        st.write("**Generated Values:**")
        st.write(f"Sensor 1: {st.session_state['generated_values'][0]:.2f}")
        st.write(f"Sensor 2: {st.session_state['generated_values'][1]:.2f}")
        st.write(f"Sensor 3: {st.session_state['generated_values'][2]:.2f}")
        st.write(f"Operational Hours: {st.session_state['generated_values'][3]:.2f}")
        if st.button('Use Generated Values'):
            st.session_state['input_features'] = st.session_state['generated_values']
              st.success("Generated values have been used. Navigate to the Results page to see the predictions.")
    st.markdown("**Or manually input values:**")
```

```
    sensor_1 = st.slider('Sensor 1', float(data['sensor_1'].min()), float(data['sensor_1'].max()), float(data['sensor_1'].mean()))
    sensor_2 = st.slider('Sensor 2', float(data['sensor_2'].min()), float(data['sensor_2'].max()), float(data['sensor_2'].mean()))
    sensor_3 = st.slider('Sensor 3', float(data['sensor_3'].min()), float(data['sensor_3'].max()), float(data['sensor_3'].mean()))
     operational_hours = st.slider('Operational Hours', int(data['operational_hours'].min()), int(data['operational_hours'].max()), int(data['operational_hours'].mean()))
  if st.button('Submit'):
    st.session_state['input_features'] = [sensor_1, sensor_2, sensor_3, operational_hours]
      st.success("Input data submitted successfully! Navigate to the Results page to see the predictions.")
elif selected == "Results":
  st.title(" 📊 Prediction Results")
  if 'input_features' not in st.session_state:
    st.warning("Please input data first in the 'Input Data' section.")
  else:
    input_features = st.session_state['input_features']
    prediction = predict_maintenance(input_features)
    st.write(f"**Remaining Useful Life (RUL):** {prediction['RUL Prediction']:.2f} hours")
    st.write(f"**Maintenance Status:** {prediction['Maintenance Prediction']}")
    st.write(f"**Anomaly Detection:** {prediction['Anomaly Detection']}")
    if prediction['Maintenance Prediction'] == 'Needs Maintenance':
      st.error(' ⚠️ Maintenance is required!')
```

A.2. SCREENSHOTS

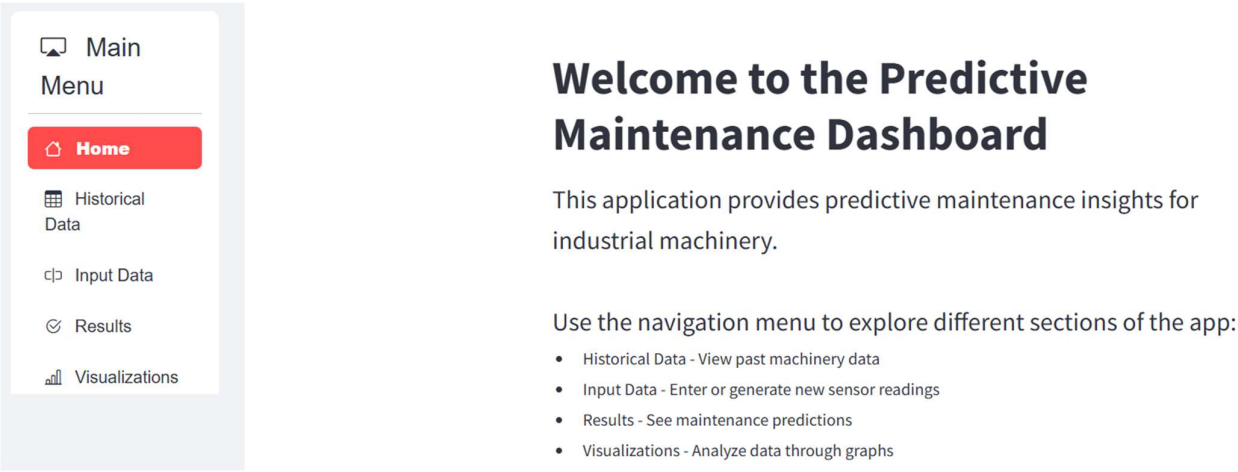
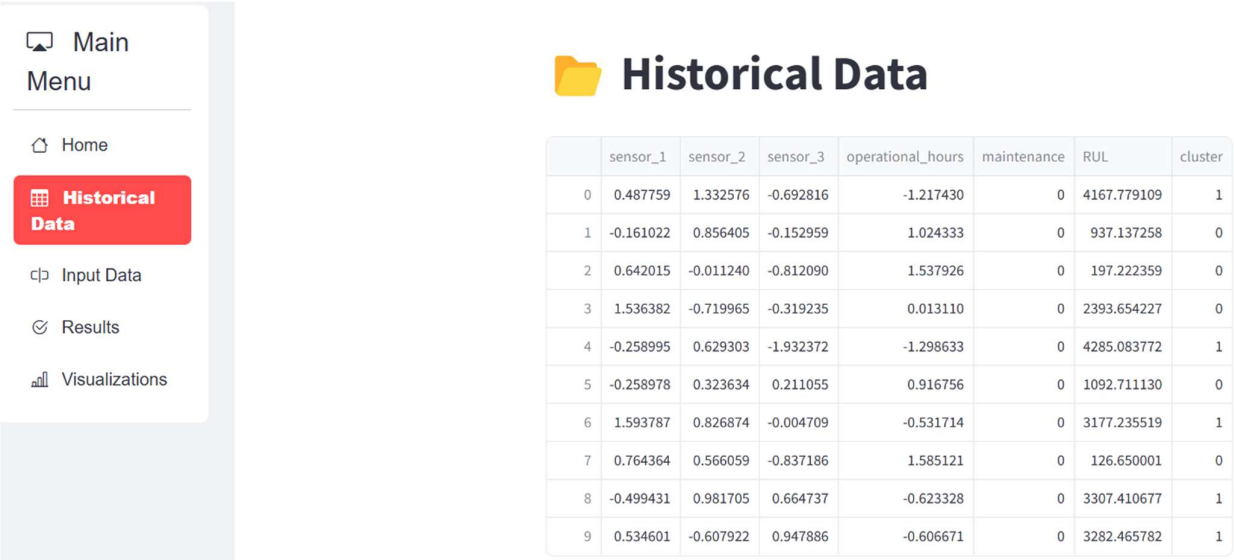


Fig A.2.1 Home Page



	sensor_1	sensor_2	sensor_3	operational_hours	maintenance	RUL	cluster
0	0.487759	1.332576	-0.692816	-1.217430	0	4167.779109	1
1	-0.161022	0.856405	-0.152959	1.024333	0	937.137258	0
2	0.642015	-0.011240	-0.812090	1.537926	0	197.222359	0
3	1.536382	-0.719965	-0.319235	0.013110	0	2393.654227	0
4	-0.258995	0.629303	-1.932372	-1.298633	0	4285.083772	1
5	-0.258978	0.323634	0.211055	0.916756	0	1092.711130	0
6	1.593787	0.826874	-0.004709	-0.531714	0	3177.235519	1
7	0.764364	0.566059	-0.837186	1.585121	0	126.650001	0
8	-0.499431	0.981705	0.664737	-0.623328	0	3307.410677	1
9	0.534601	-0.607922	0.947886	-0.606671	0	3282.465782	1

Fig A.2.2 Sample Dataset

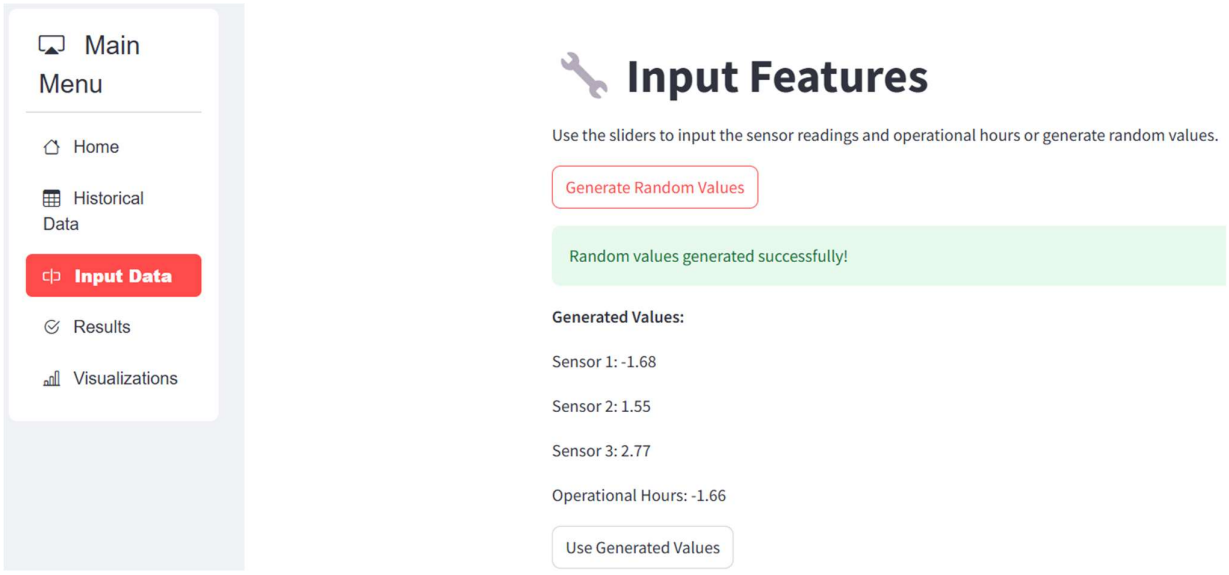


Fig A.2.3 Random Input Generation



Fig A.2.4 User Inputs

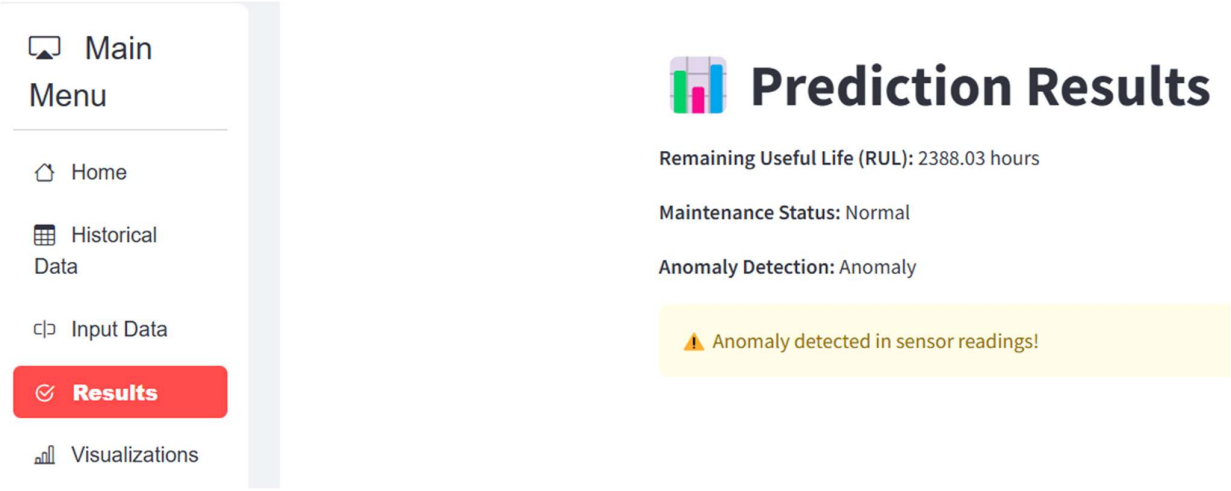


Fig A.2.5 Prediction with Anomaly

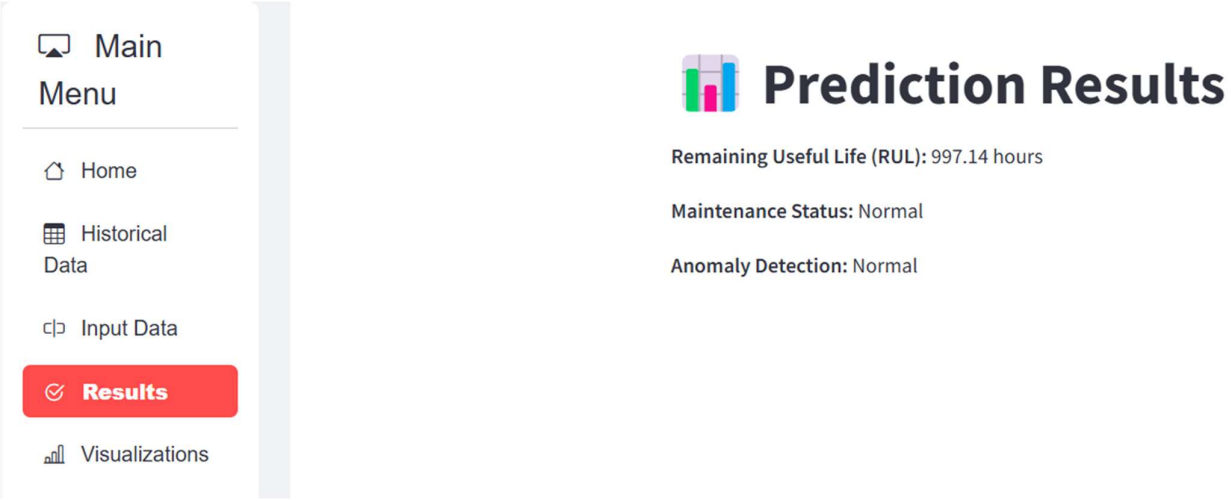


Fig. A.2.6 Prediction without Anomaly

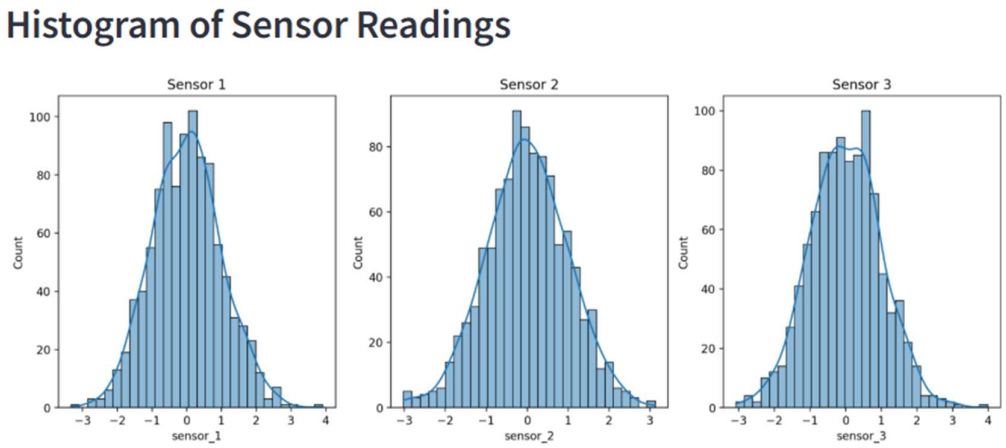


Fig. A.2.7 Histogram chart for Sensor readings

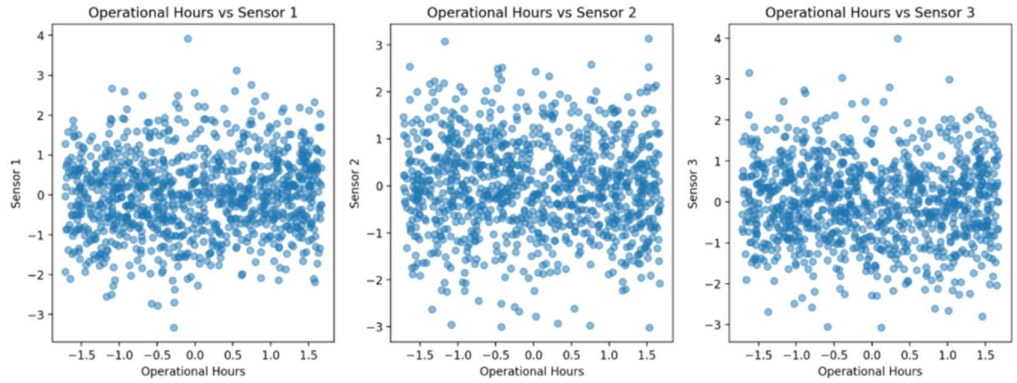


Fig. A.2.8 Scatter Plot for Sensor Readings vs Operational Hours

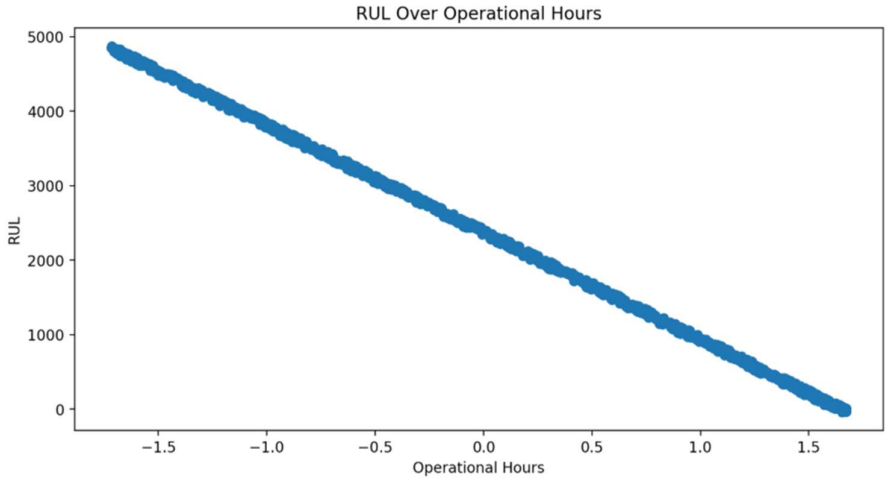


Fig. A.2.9 Line chart for RUL vs Operational Hours

REFERENCES

1. Paolanti, M., Romeo, L., Felicetti, A., Mancini, A., & Frontoni, E. (2018). Machine Learning approach for Predictive Maintenance in Industry 4.0. 2018 14th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA),
2. Abbasi, J. A. (2021). Predictive Maintenance in Industrial Machinery using Machine Learning. Master's Thesis, Luleå University of Technology.
3. Khan, S., Yairi, T. (2018). A review on the application of deep learning in system health management. Mechanical Systems and Signal Processing, 107, 241–265.
4. Zonta, T., da Costa, C. A., da Rosa Righi, R., de Lima, M. J., da Trindade, E. S., & Li, G. P. (2020). Predictive maintenance in the Industry 4.0: A systematic literature review. Computers & Industrial Engineering, 150, 106889.
5. Sang, G. M., Xu, L., & de Vrieze, P. (2021). A Predictive Maintenance Model for Flexible Manufacturing in the Context of Industry 4.0. Frontiers in Big Data, 4, 663466.
6. Kumar, A., & Singh, H. (2023). Predictive Maintenance using Machine Learning: A Case Study in Manufacturing Management. 2023 IEEE International Conference on Industrial Engineering and Engineering Management , 1–6.
7. Kumar, A., & Singh, H. (2022). Predictive Maintenance Planning for Industry 4.0 Using Machine Learning. Sustainability, 14(6), 3387.
8. Nangia, S., Makkar, S., & Hassan, R. (2020). IoT based Predictive Maintenance in Manufacturing Sector. Proceedings of the International Conference on Innovative Computing & Communications (ICICC) 2020.
9. Cummins, L., Sommers, A., Bakhtiari Ramezani, S., Mittal, S., Jabour, J., Seale, M., & Rahimi, S. (2023). Explainable Predictive Maintenance: A Survey of Current Methods, Challenges, and Opportunities. arXiv preprint arXiv:2401.07871.
10. Hamaide, V., Joassin, D., Castin, L., & Glineur, F. (2022). A two-level machine learning framework for predictive maintenance: comparison of learning formulations. arXiv preprint arXiv:2204.10083.
11. Zhang, W., Yang, D., & Wang, H. (2019). Data-Driven Methods for Predictive Maintenance of Industrial Equipment: A Survey. IEEE Systems Journal, 13(3), 2213–2227.

12. Lei, Y., Li, N., Guo, L., Li, N., Yan, T., & Lin, J. (2020). Machinery Health Prognostics: A Systematic Review from Data Acquisition to RUL Prediction. Mechanical Systems and Signal Processing, 104761.
13. Goyal, D., & Pabla, B. S. (2016). Condition Based Maintenance of Machine Tools—A Review. CIRP Journal of Manufacturing Science and Technology, 10, 24–35.
14. Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., & Beghi, A. (2015). Machine Learning for Predictive Maintenance: A Multiple Classifier Approach. IEEE Transactions on Industrial Informatics, 11(3), 812–820.
15. Bousdekis, A., Magoutas, B., Apostolou, D., & Mentzas, G. (2019). A Decision Support Framework for Predictive Maintenance Based on Data-Driven and Expert-Based Reasoning. Journal of Intelligent Manufacturing, 30, 1177–1192.